University of Duisburg-Essen
Networked Embedded Systems Group
Institute for Computer Science and
Business Information Systems (ICB)
Schützenbahn 70
D-45127 Essen, Germany

Evaluierung des Einflusses der Semantik auf die Verbesserung von Merkmalsextraktoren und Matchern

Bachelorarbeit

MHD Jehad Saloum

Universität Duisburg-Essen | Networked Embedded Systems Group

Betreuer: Theodor Josef Vogl

Thema der Arbeit

Worum geht es konkret?

Feature Matching

- Lokale Bildpunkte werden detektiert [1]
- Jeder Punkt erhält einen Deskriptor
- Zwischen zwei Bildern werden Matches gesucht

Geometrische Bewertung

- Prüfung mit Homographie [2] und RANSAC [3]
- Ein Match ist korrekt, wenn er zum geometrischen Modell passt

Semantische Filterung

- Prüfung über Objektregionen und Bounding Boxes [4]
- Ein Match soll inhaltlich zur gleichen Objektregion gehören

Wie verändert sich die Bewertung, wenn geometrische Matches zusätzlich semantisch gefiltert werden?

Motivation und Problem

Geometrisch korrekt heißt nicht automatisch semantisch korrekt.

Motivation

- Feature Matching ist unter anderem die Grundlage für visuelle Lokalisation, 3D-Rekonstruktion [5], Augmented Reality [6] und SLAM [7]
- Robuste Korrespondenzen ermöglichen räumliche Interpretation statt einfachem Pixelvergleich
- Qualität und Laufzeit sind für praktische Vision-Systeme entscheidend

Problemstellung

- Viele Verfahren bewerten Matches hauptsächlich geometrisch
- Wiederholende Strukturen können geometrisch plausibel, aber semantisch falsch sein
- Der semantische Kontext wird in klassischen Benchmarks wie HPatches [8] kaum berücksichtigt

Ziel und Beitrag der Arbeit

Systematischer Vergleich ohne und mit semantischer Filterung.

Ziele

- Sechs Feature-Detektoren vergleichen
- Fünf Matching-Verfahren vergleichen
- Laufzeit, Keypoint-Anzahl und Qualitätsmetriken analysieren
- Geometrische Bewertung mit semantischer Filterung gegenüberstellen

Beitrag

- Einheitliche Evaluierungspipeline
- Wiederverwendung gleicher Extraktionsdaten
- Semantische Prüfung über Bounding-Box-Annotationen
- Direkter Vergleich aller Detektor-Matcher-Kombinationen

Verglichene Verfahren

- Detektoren: SIFT [1], ORB [9], BRISK [10], SuperPoint [11], DISK [12], KeyNetHardNet [13,14]
- Matcher: BruteForce [20], KNN+Lowe [1], FLANN [15], FAISS [16], CrossCheck [20]

Von Keypoints zu Matches

Die Grundpipeline der Merkmalszuordnung.

1 Detektion

- Stabile Merkmale finden: Ecken, Kanten und markante Strukturen

→

2 Beschreibung

- Umgebung jedes Keypoints als Deskriptor codieren

→

3 Matching

- Ähnliche Deskriptoren zwischen zwei Bildern zuordnen

→

4 Bewertung

- Geometrisch: Homographie/RANSAC [2,3]
- Semantisch: Objektregion/Bounding Box [4]

Datensatz und Bewertungsgrundlage

HPatches als geometrischer Benchmark, ergänzt durch semantische Annotationen.

HPatches

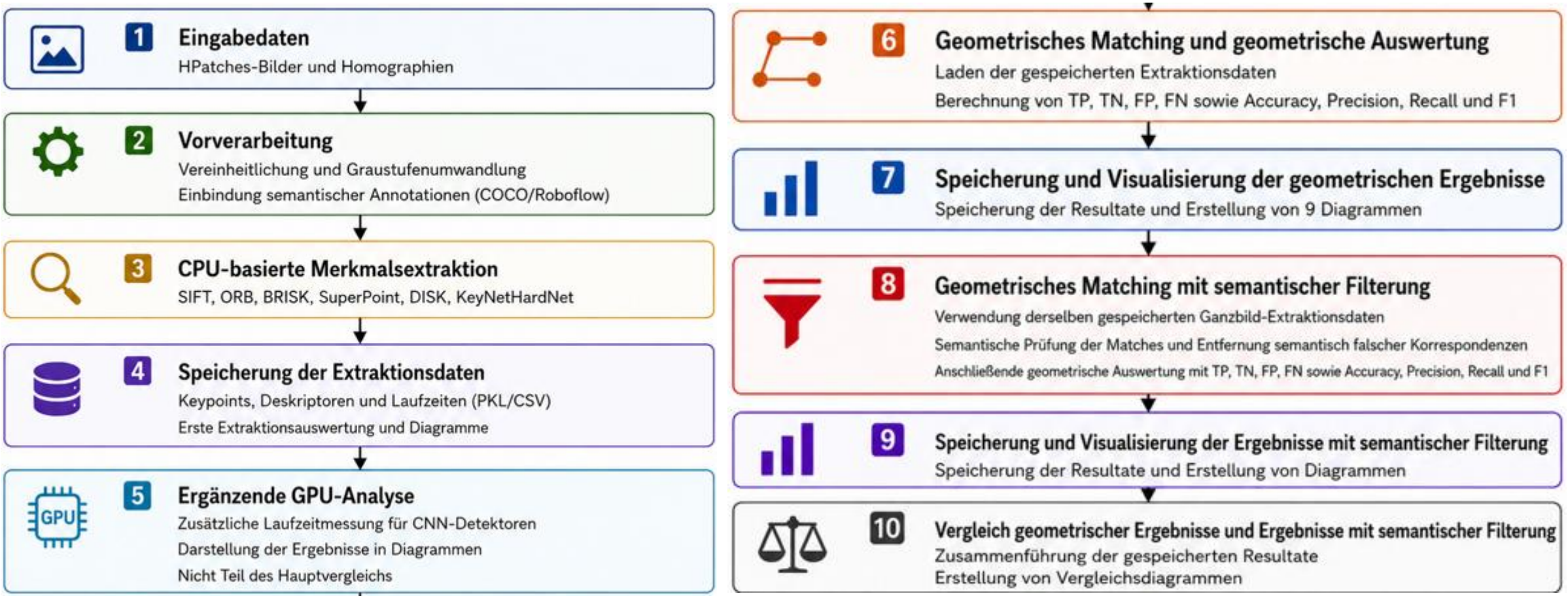
- 116 Bildsequenzen [8]
- Jede Szene besteht aus 6 Bildern
- Bild 1 wird mit Bild 2- 6 verglichen
- Homographien dienen als geometrische Ground Truth
- Sequenzen mit Viewpoint- und Illumination-Änderungen

Semantik und Metriken

- Objektbezogene Bounding Boxes im COCO-Format [4]
- Prüfung, ob beide Matchpunkte zur gleichen Objektregion gehören
- Metriken: Precision, Recall, F1-Score, Accuracy [18]
- Effizienz: Keypoint-Anzahl und Laufzeit in ms

Arbeitsablauf der Evaluierungspipeline

Von Eingabedaten bis zum Vergleich der Ergebnisse.



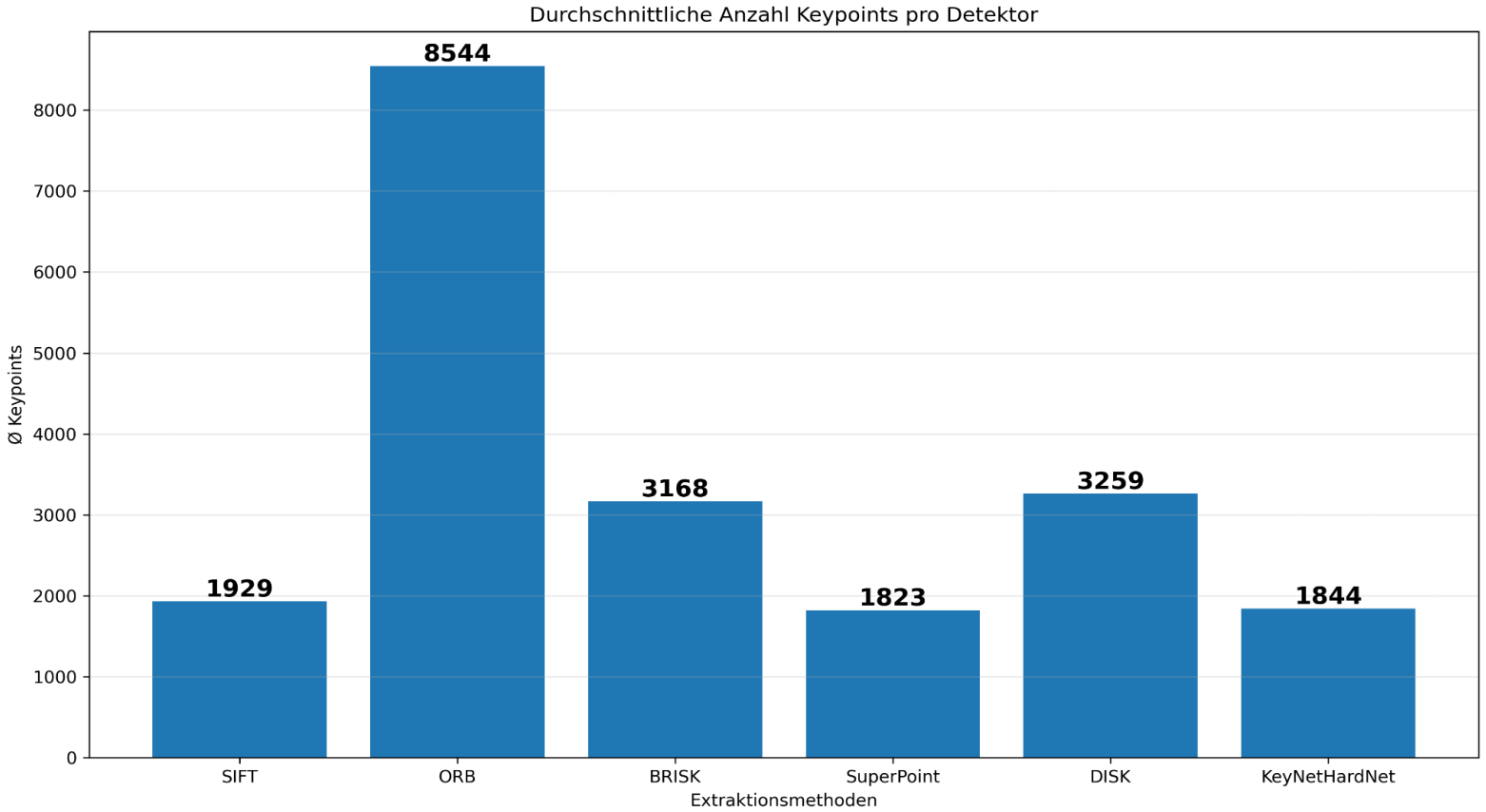
Experimentaufbau

Vier Experimente strukturieren die Evaluation.

Experiment 1	**CPU-basierte Merkmalsextraktion**	Keypoints und Extraktionszeit für alle Detektoren
Experiment 2	**GPU-Zusatzanalyse**	Beschleunigung der CNN-basierten Verfahren
Experiment 3	**Geometrisches Matching**	Bewertung mit Homographie und RANSAC
Experiment 4	**Geometrisch + Semantik**	Semantische Filterung, danach erneute geometrische Bewertung

Experiment 1: Keypoint-Anzahl

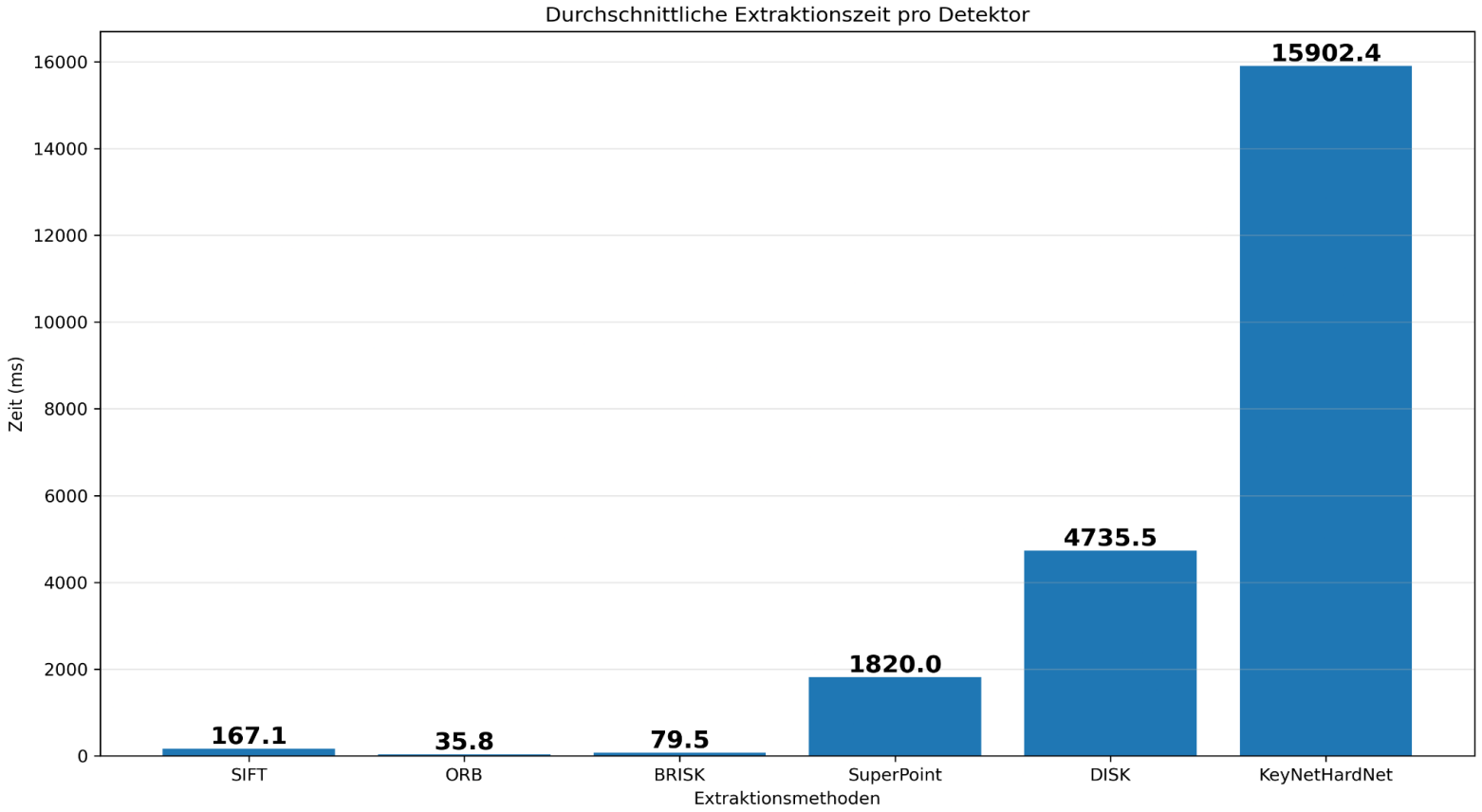
CPU-basierte Merkmalsextraktion



- ORB liefert die meisten Keypoints (Ø 8544)
- DISK und BRISK liegen im mittleren Bereich
- Viele Keypoints erhöhen die Matching-Chancen, aber auch den Aufwand

Experiment 1: CPU-Extraktionszeit

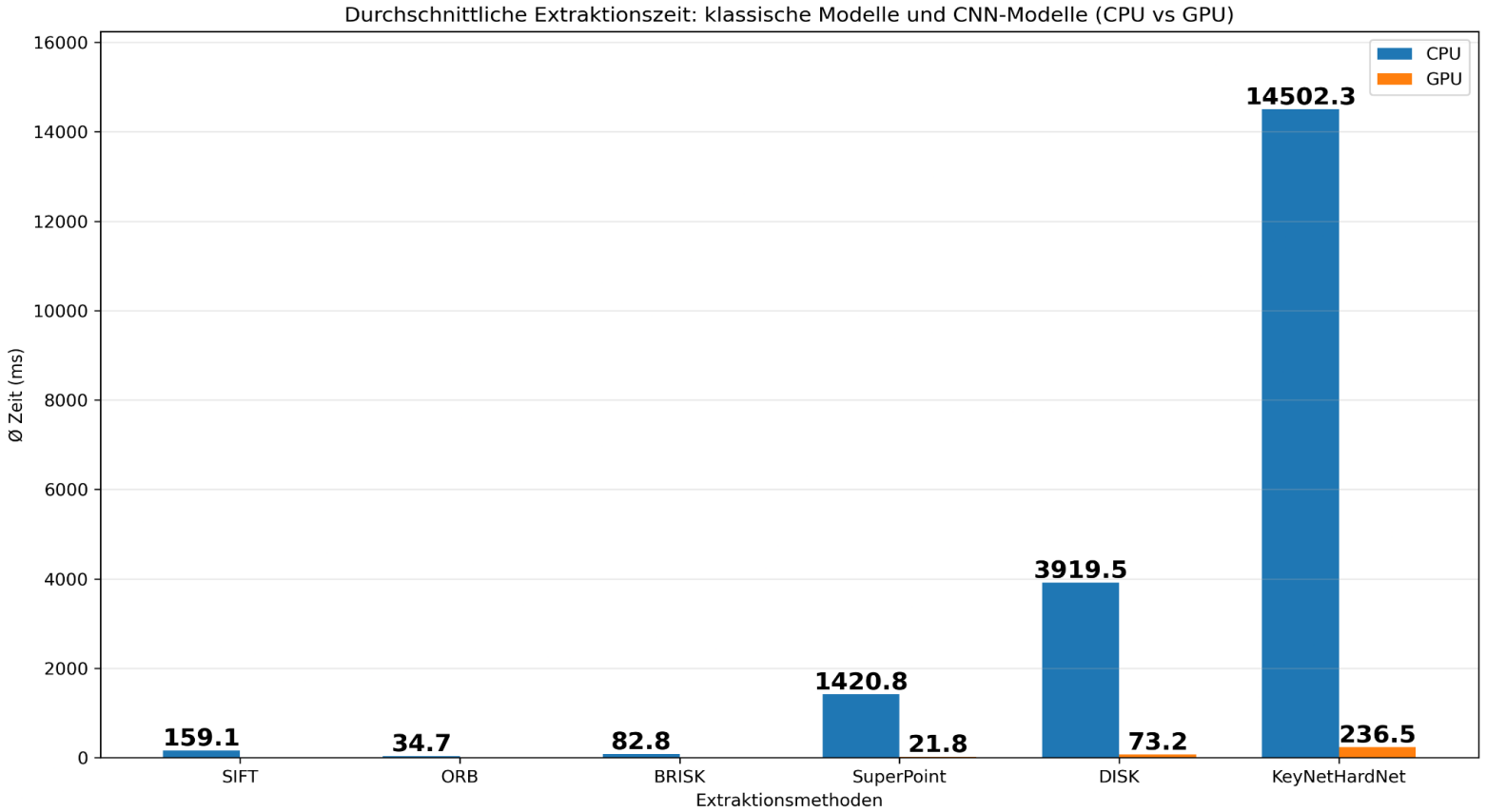
Laufzeitvergleich der Detektoren



- ORB ist auf der CPU am schnellsten (35,8 ms)
- BRISK und SIFT bleiben ebenfalls effizient
- CNN-Verfahren sind auf der CPU deutlich langsamer

Experiment 2: CPU vs. GPU

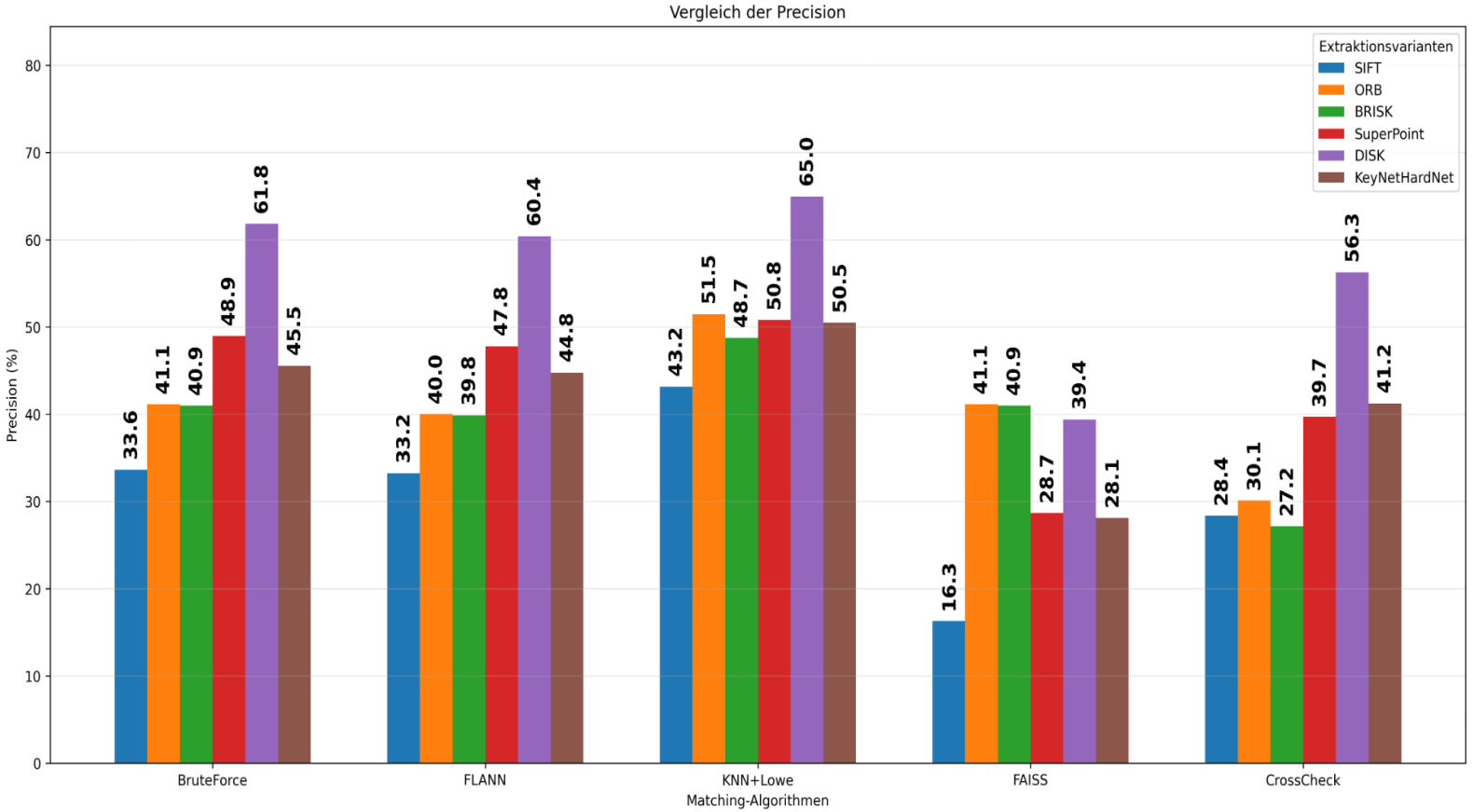
GPU-Zusatzanalyse der CNN-basierten Verfahren



- CNN-Verfahren profitieren stark von der GPU
- SuperPoint: ca. 65x schneller
- DISK: ca. 54x schneller
- KeyNetHardNet: ca. 61x schneller
- Insgesamt: rund 98 % Zeitreduktion

Experiment 3: Precision

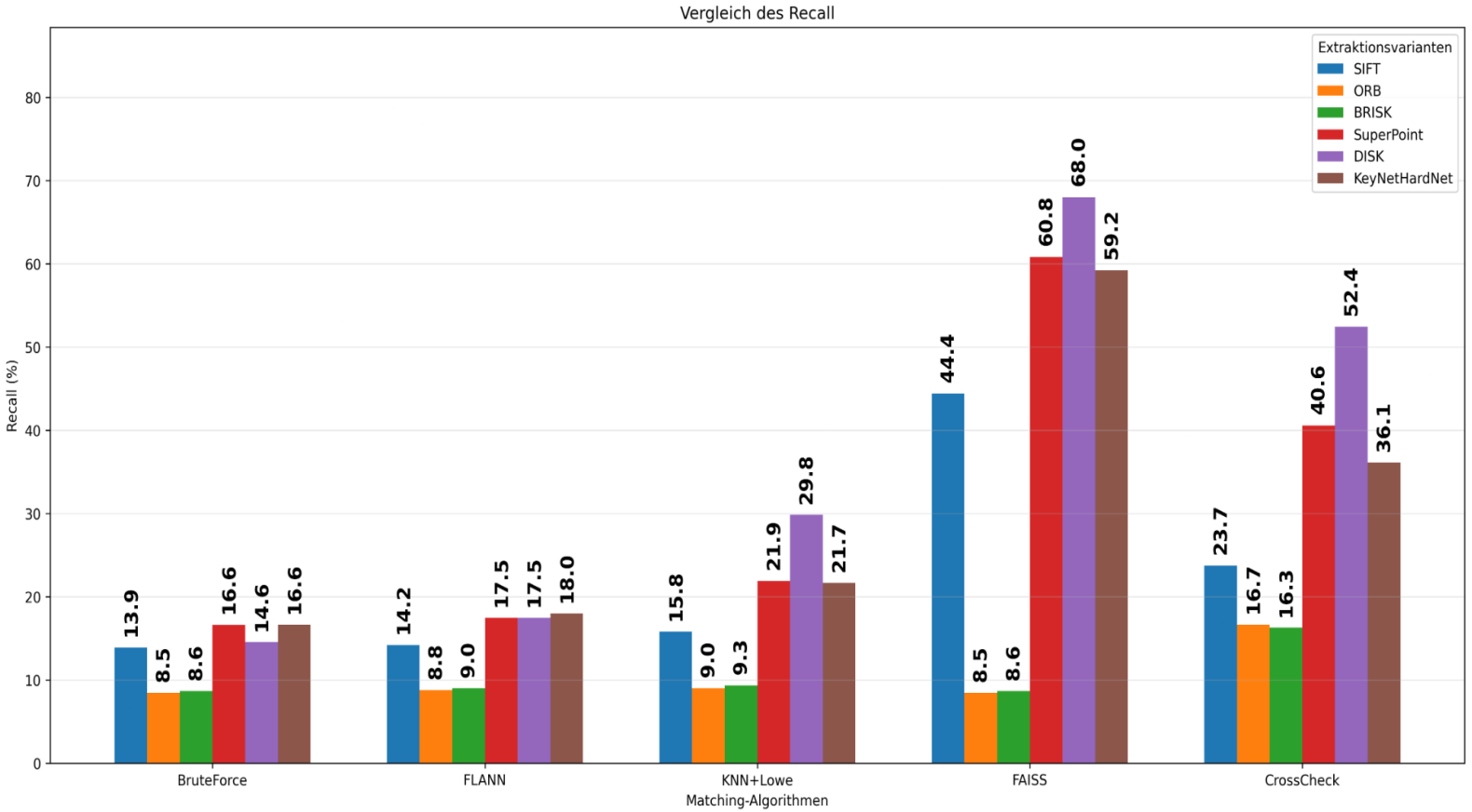
Geometrisches Matching



- DISK erreicht häufig die höchste Precision
- Bestwert: DISK + KNN+Lowe mit 65,0 %
- Hohe Precision bedeutet: weniger falsche Matches

Experiment 3: Recall

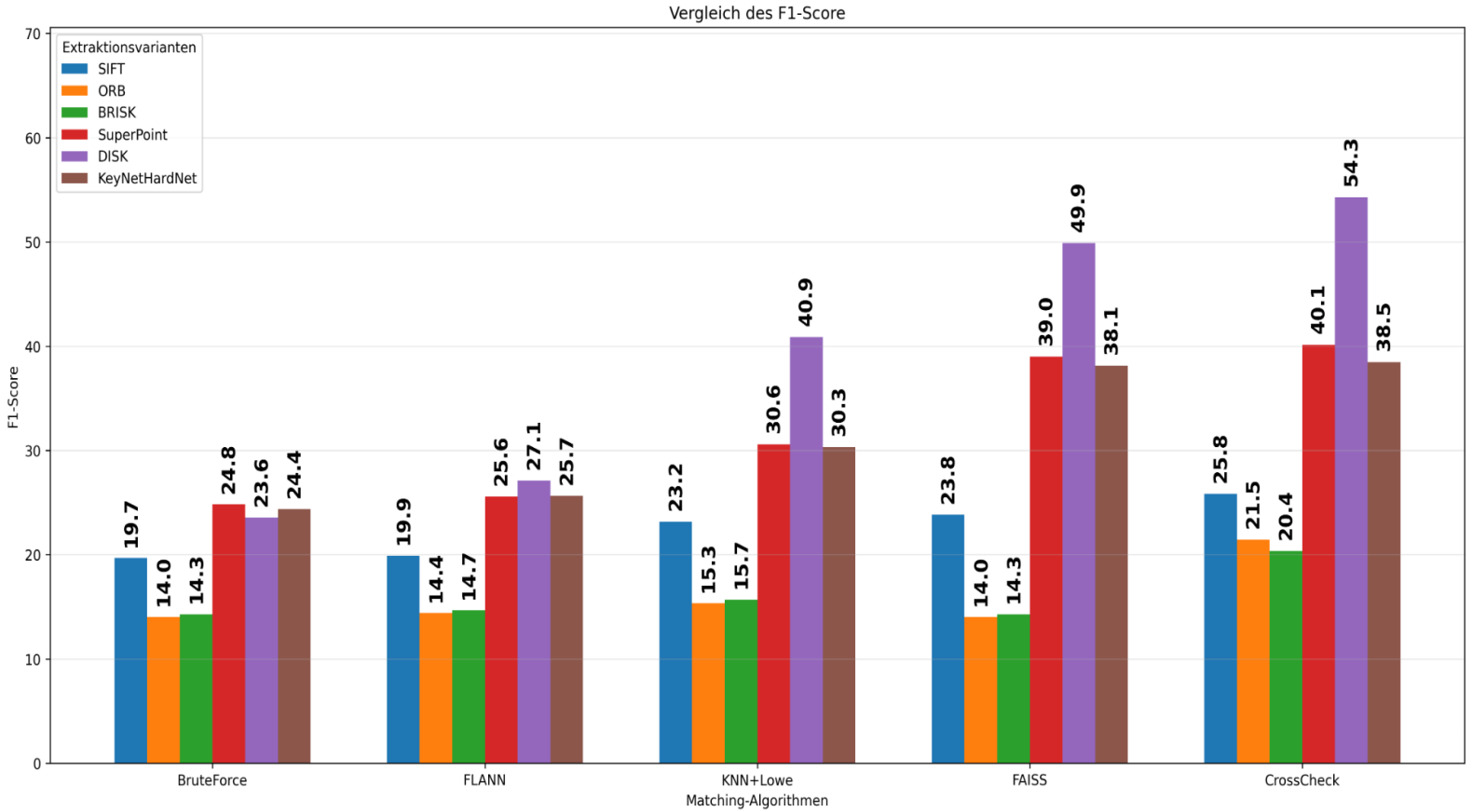
Geometrisches Matching



- FAISS erreicht die höchsten Recall-Werte
- Besonders stark: DISK, SuperPoint und KeyNetHardNet
- Hoher Recall bedeutet: viele korrekte Matches werden gefunden

Experiment 3: F1-Score

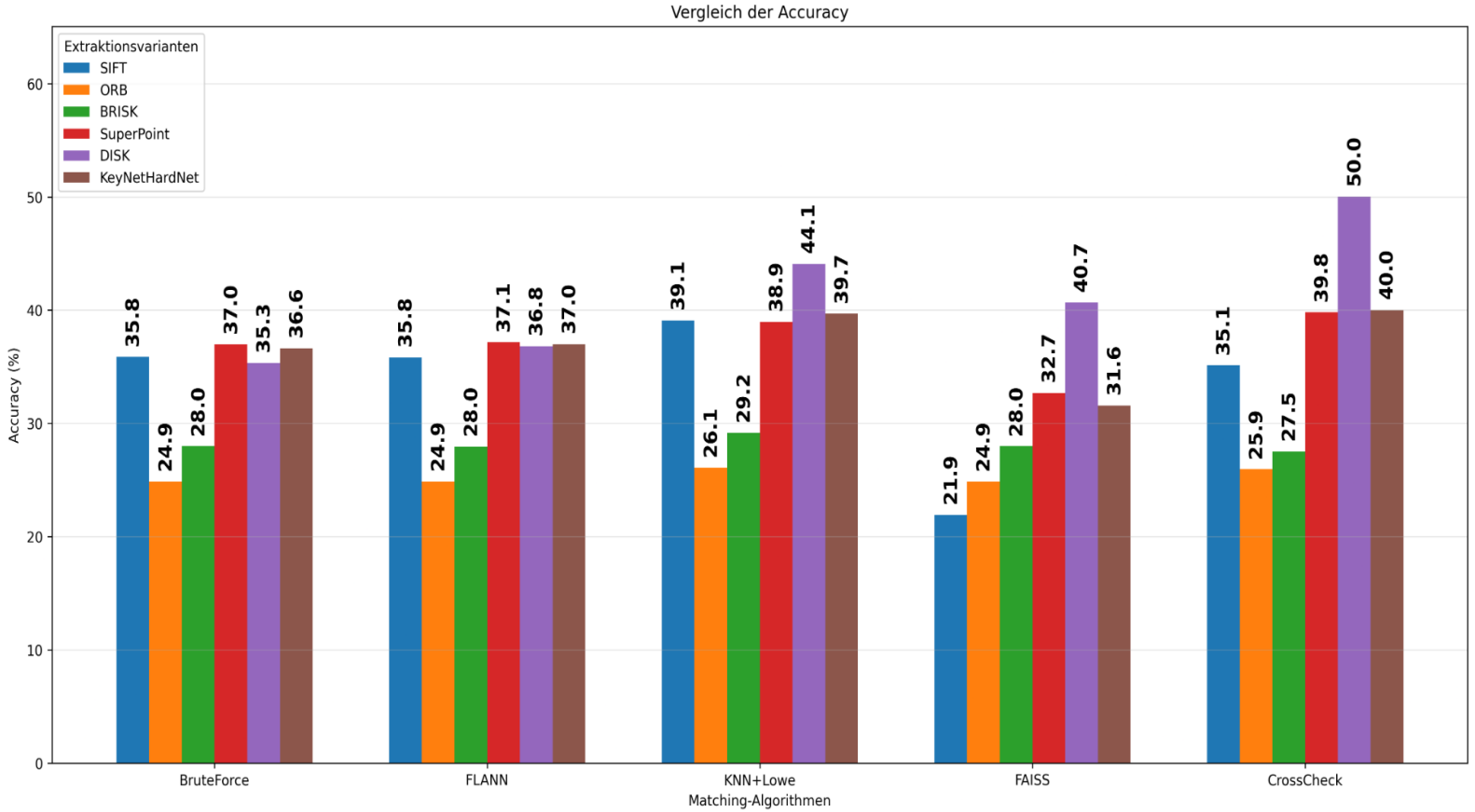
Geometrisches Matching



- DISK zeigt die beste Balance aus Precision und Recall
- Höchster Wert: DISK + CrossCheck mit 54,3 %
- Lernbasierte Detektoren schneiden meist stärker ab

Experiment 3: Accuracy

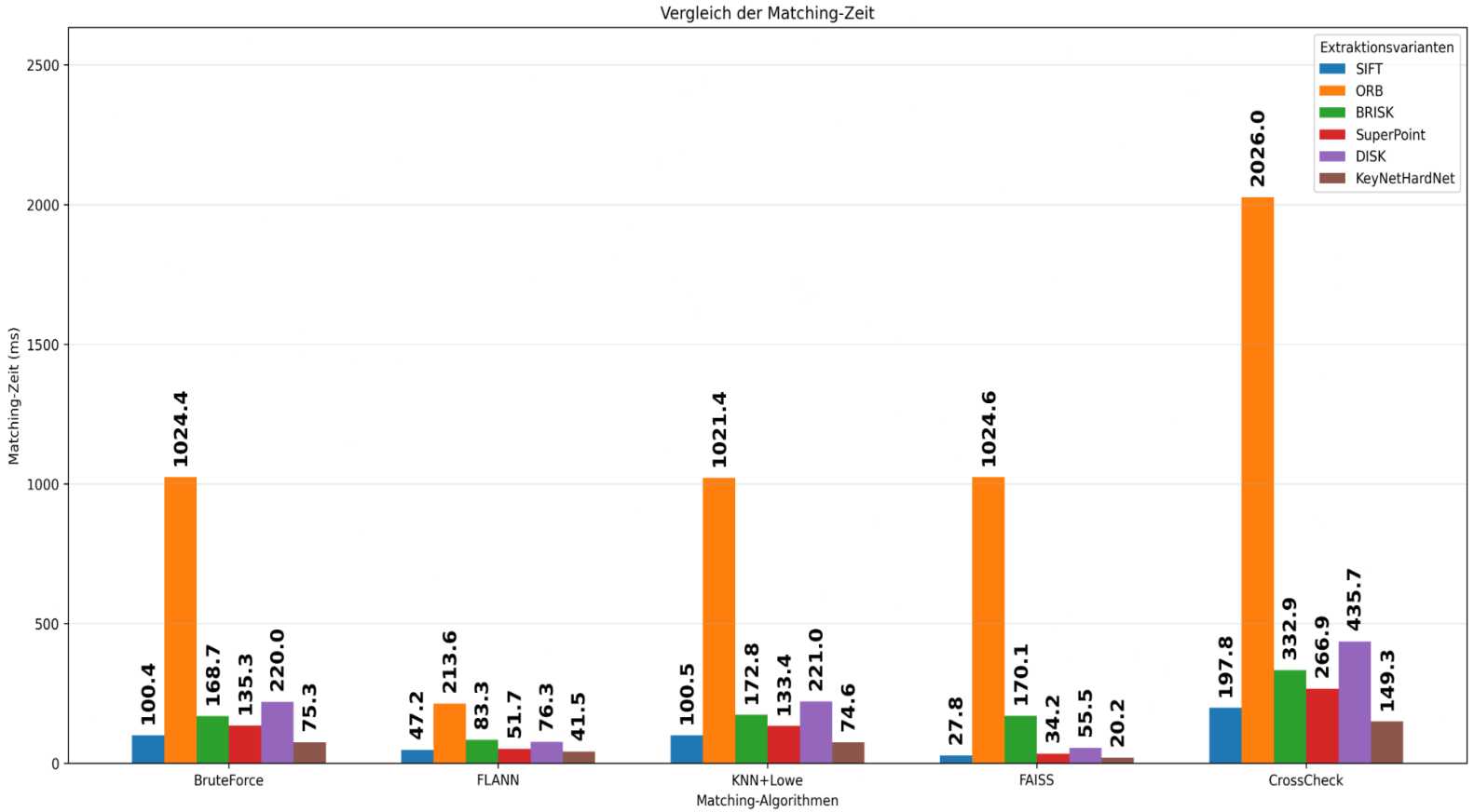
Geometrisches Matching



- Höchster Wert: DISK + CrossCheck mit 50,0 %
- CNN-basierte Verfahren dominieren die besten Kombinationen
- Accuracy allein reicht nicht für die Gesamtbewertung

Experiment 3: Matching-Laufzeit

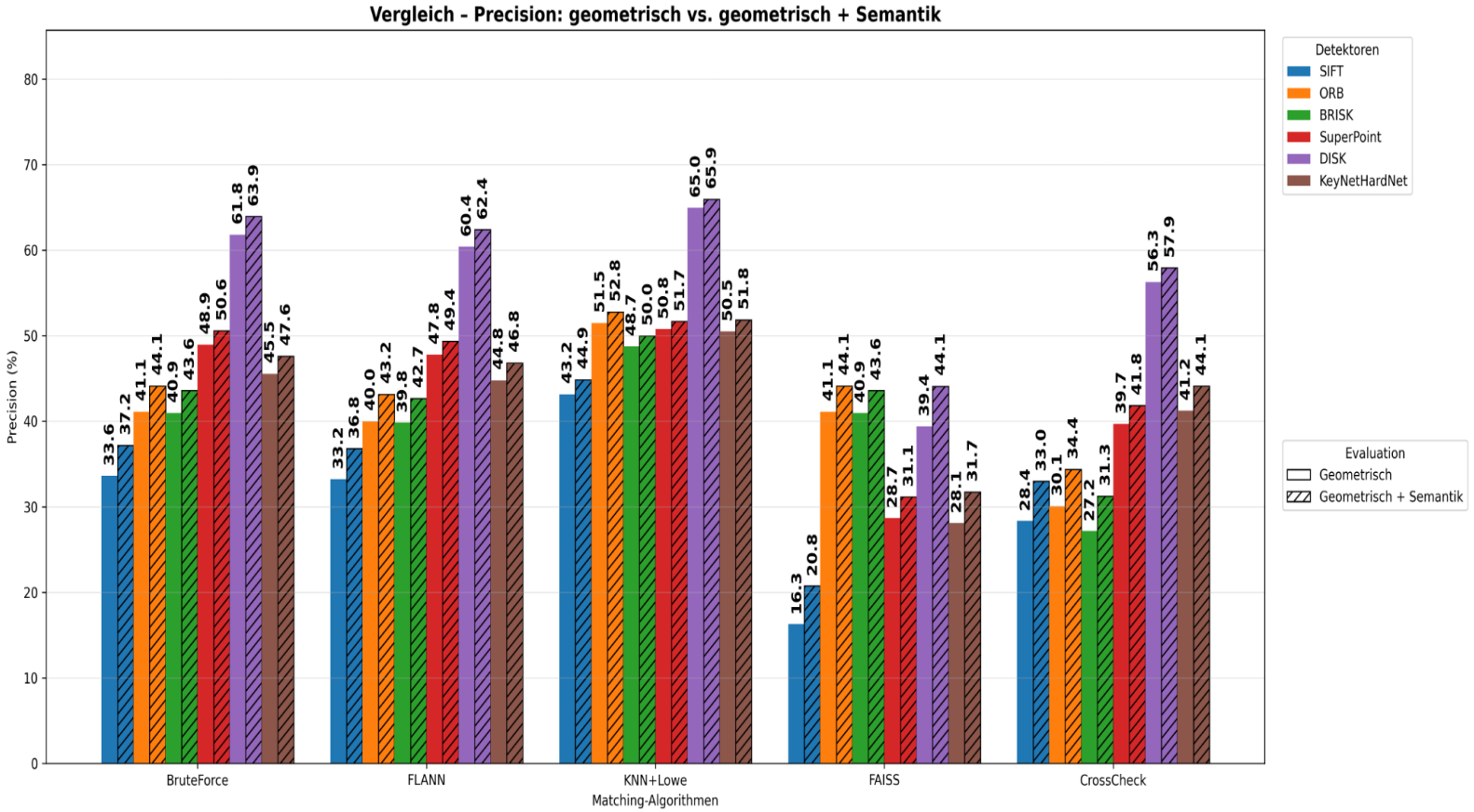
Laufzeit der Matcher



- FLANN ist insgesamt am schnellsten
- CrossCheck ist am langsamsten, besonders bei ORB
- Schnelle Matcher sind nicht automatisch qualitativ die besten

Experiment 4: Precision mit Semantik

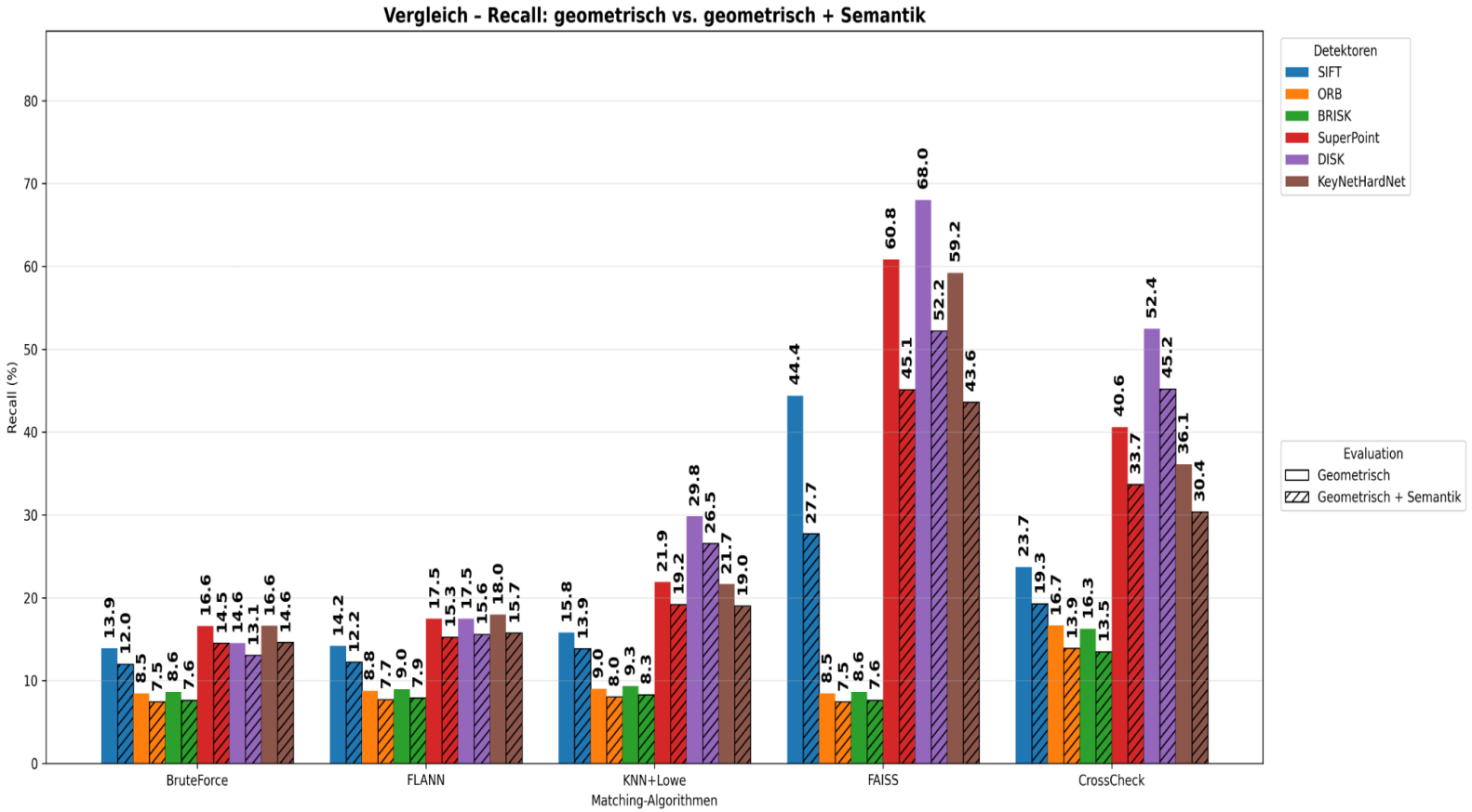
Geometrisch vs. geometrisch + Semantik



- Precision steigt in allen 30 Kombinationen
- Semantik entfernt viele semantisch falsche Matches
- Die Matchliste wird dadurch sauberer

Experiment 4: Recall mit Semantik

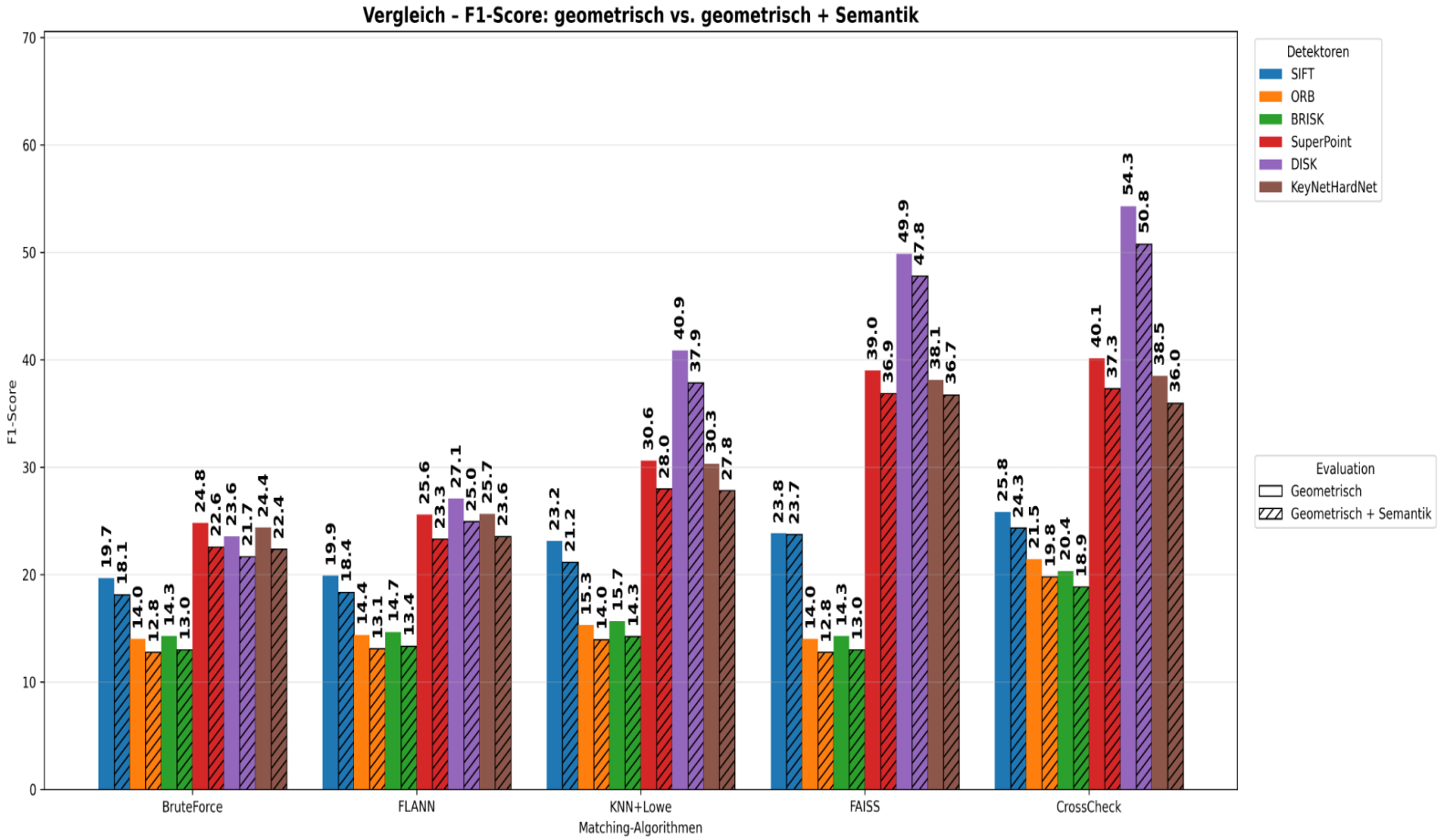
Geometrisch vs. geometrisch + Semantik



- Recall sinkt in allen Kombinationen
- Es werden auch einige geometrisch korrekte Matches entfernt
- Dadurch sinkt auch der F1-Score in allen Kombinationen

Experiment 4: F1-Score mit Semantik

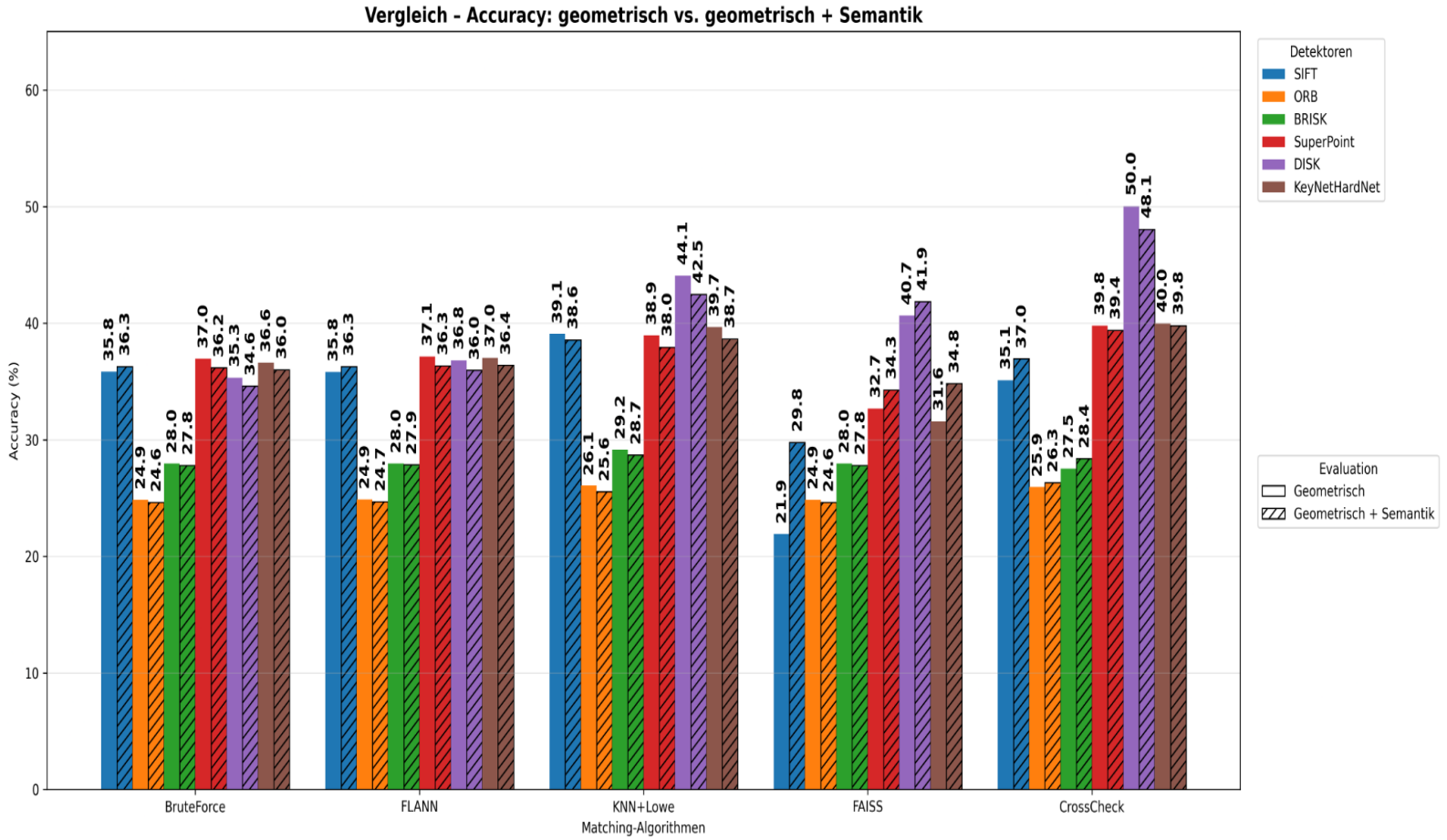
Geometrisch vs. geometrisch + Semantik



- Der F1-Score sinkt in allen Kombinationen
- Grund: Der Recall-Verlust ist stärker als der Precision-Gewinn
- Semantik verbessert deshalb nicht automatisch die Gesamtbalance

Experiment 4: Accuracy mit Semantik

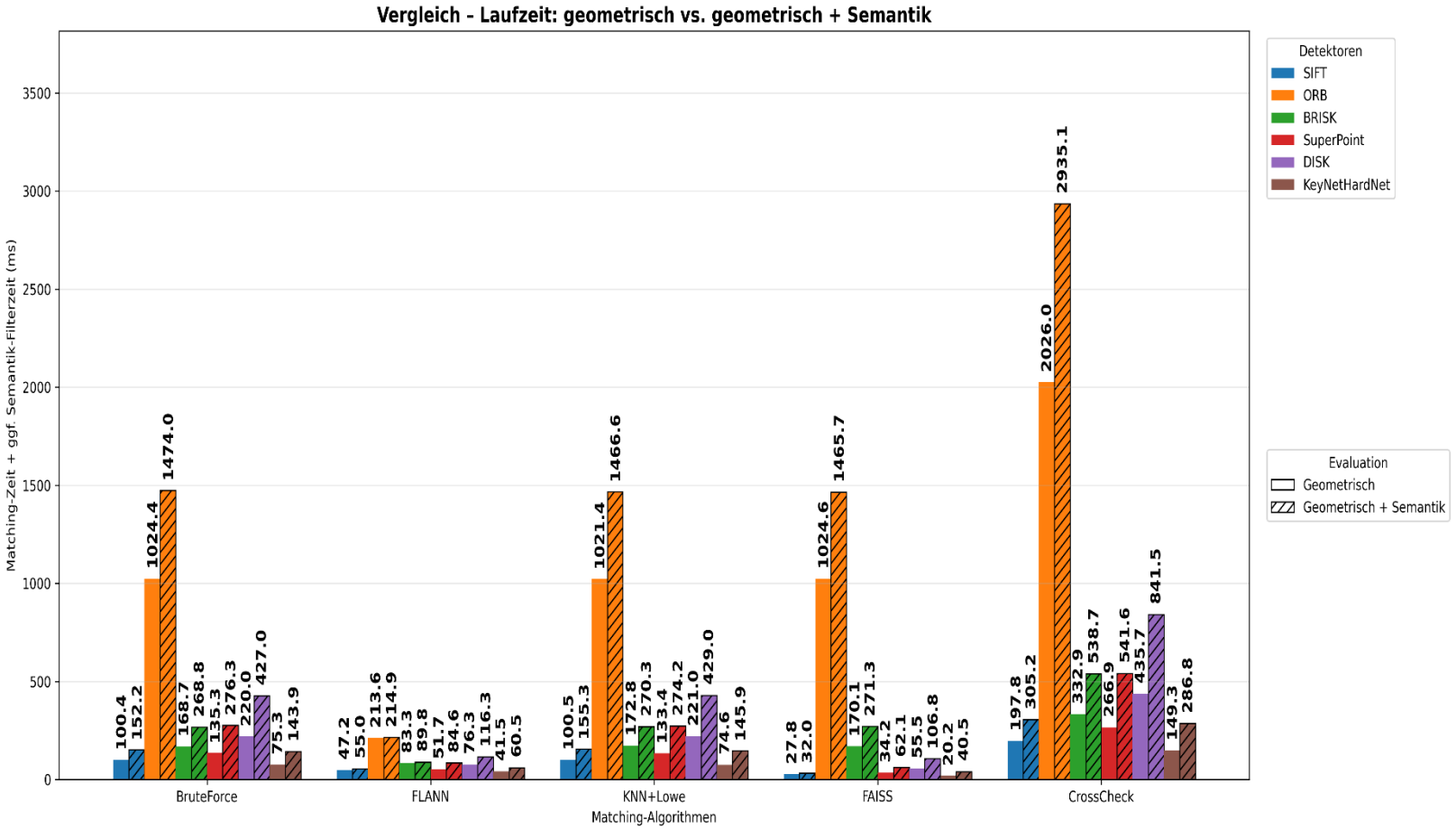
Geometrisch vs. geometrisch + Semantik



- Accuracy steigt nur in 9 von 30 Fällen
- In 21 Fällen sinkt sie nach der Filterung
- Verbesserung entsteht nur, wenn der Gewinn an TN den Verlust an TP überwiegt

Experiment 4: Zeit mit Semantik

Geometrisch vs. geometrisch + Semantik



- Zeit steigt in allen 30 Kombinationen
- Weil die semantische Filterung nach dem Matching durchgeführt wird, wird die Filterzeit zur Matching-Zeit addiert.

Fazit und Ausblick

- **Ausblick**
- Feinere Segmentierung statt Bounding Boxes
- Semantik direkt in das Matching integrieren
- Weitere Datensätze und moderne Matcher wie LightGlue [19] untersuchen

- **Fazit**
- Precision [18] steigt in allen Kombinationen
- Recall und F1 [18] sinken in allen Kombinationen
- Accuracy [18] steigt nur in 9 von 30 Fällen
- Semantik wirkt vor allem als Präzisionsfilter

Literatur

[1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” International Journal of Computer Vision, 2004.

[2] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004.

[3] M. A. Fischler and R. C. Bolles, “Random Sample Consensus,” Communications of the ACM, 1981.

[4] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” ECCV, 2014.

[5] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” CVPR, 2016.

[6] É. Marchand, H. Uchiyama, and F. Spindler, “Pose Estimation for Augmented Reality,” IEEE TVCG, 2016.

[7] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2,” IEEE Transactions on Robotics, 2017.

[8] V. Balntas et al., “HPatches: A Benchmark and Evaluation of Handcrafted and Learned Local Descriptors,” CVPR, 2017.

[9] E. Rublee et al., “ORB: An Efficient Alternative to SIFT or SURF,” ICCV, 2011.

[10] S. Leutenegger, M. Chli, and R. Y. Siegwart, “BRISK,” ICCV, 2011.

[11] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint,” CVPR Workshops, 2018.

[12] M. J. Tyszkiewicz, P. Fua, and E. Trulls, “DISK: Learning Local Features with Policy Gradient,” NeurIPS, 2020.

[13] A. Barroso-Laguna et al., “Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters,” ICCV, 2019.

[14] A. Mishchuk et al., “Working Hard to Know Your Neighbor’s Margins,” NeurIPS, 2017.

[15] M. Muja and D. G. Lowe, “Scalable Nearest Neighbor Algorithms for High Dimensional Data,” IEEE TPAMI, 2014.

[16] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” IEEE TBD, 2019.

[17] E. Riba et al., “A Survey on Kornia: An Open Source Differentiable Computer Vision Library for PyTorch,” arXiv, 2020.

[18] D. M. W. Powers, “Evaluation: From Precision, Recall and F-Measure to ROC,” Journal of Machine Learning Technologies, 2011.

[19] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” ICCV, 2023.

[20] OpenCV, “Feature Matching,” OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html

Vielen Dank für Ihre Aufmerksamkeit

Fragen?



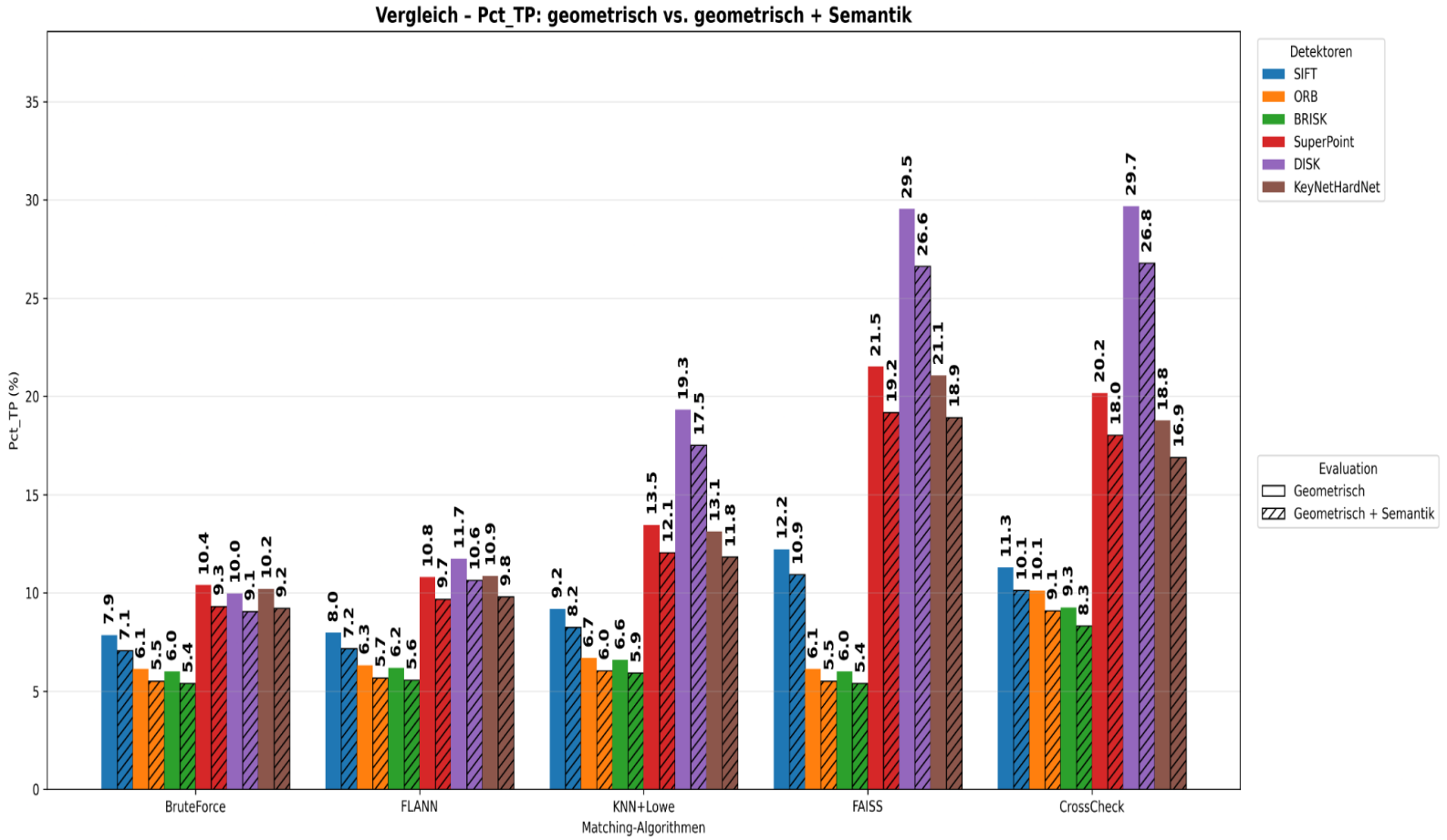
Backup:Interpretation der Metriken [18]

- Precision: Wie viele der gefundenen Matches sind korrekt?
- Recall: Wie viele der tatsächlich korrekten Matches wurden gefunden?
- F1-Score: Harmonisches Mittel aus Precision und Recall
- Accuracy: Anteil aller richtigen Entscheidungen an allen Fällen

- **Warum steigen Precision und sinken Recall/F1?**
- Die semantische Filterung entfernt viele False Positives
- Dadurch steigt die Precision
- Gleichzeitig gehen aber auch einige True Positives verloren
- Dadurch sinken Recall und oft auch F1-Score
- **Warum steigt die Accuracy nur in 9 Kombinationen?**
- In diesen neun Fällen überwiegt der Gewinn an True Negatives den Verlust an True Positives [18], wodurch die Accuracy steigt

Experiment 4: True Positives

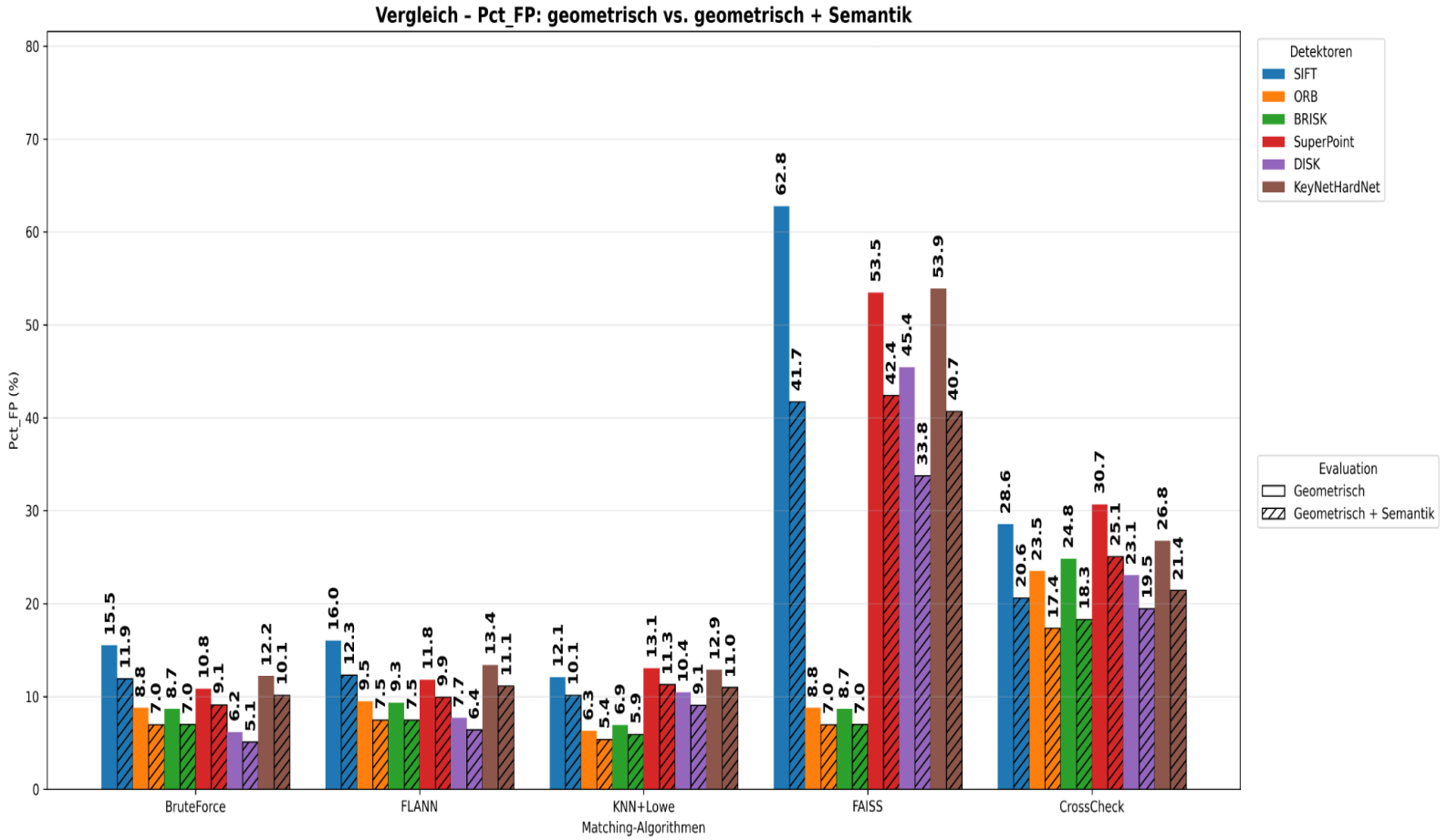
Warum sinken Recall und F1?



- Einige True Positives gehen durch die semantische Filterung verloren
- Dadurch werden weniger korrekte Matches behalten
- Das erklärt den Rückgang von Recall und F1

Experiment 4: False Positives

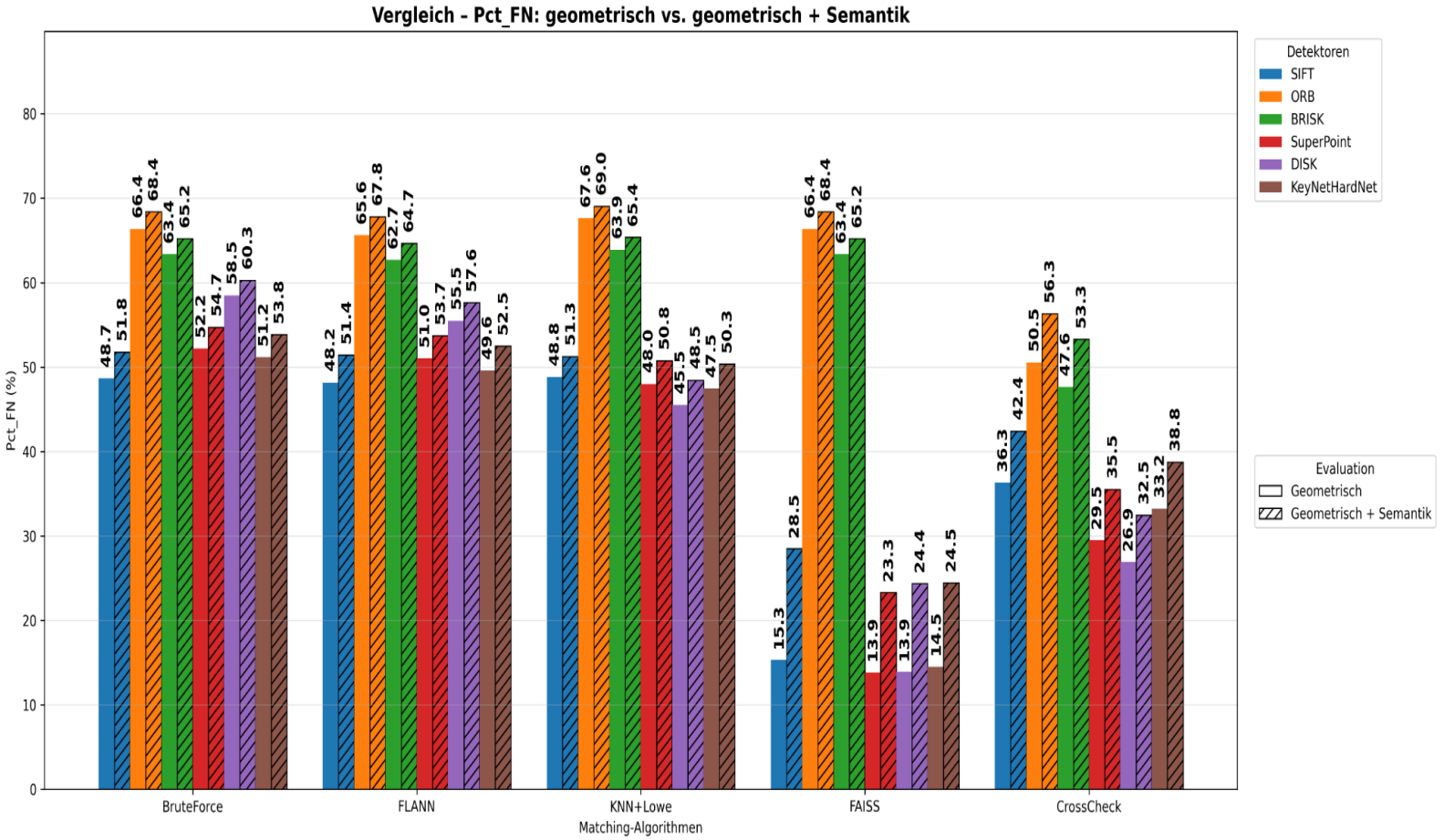
Warum steigt die Precision?



- False Positives sinken in vielen Kombinationen deutlich
- Viele falsche Matches werden entfernt
- Dadurch steigt die Precision systematisch

Experiment 4: False Negatives

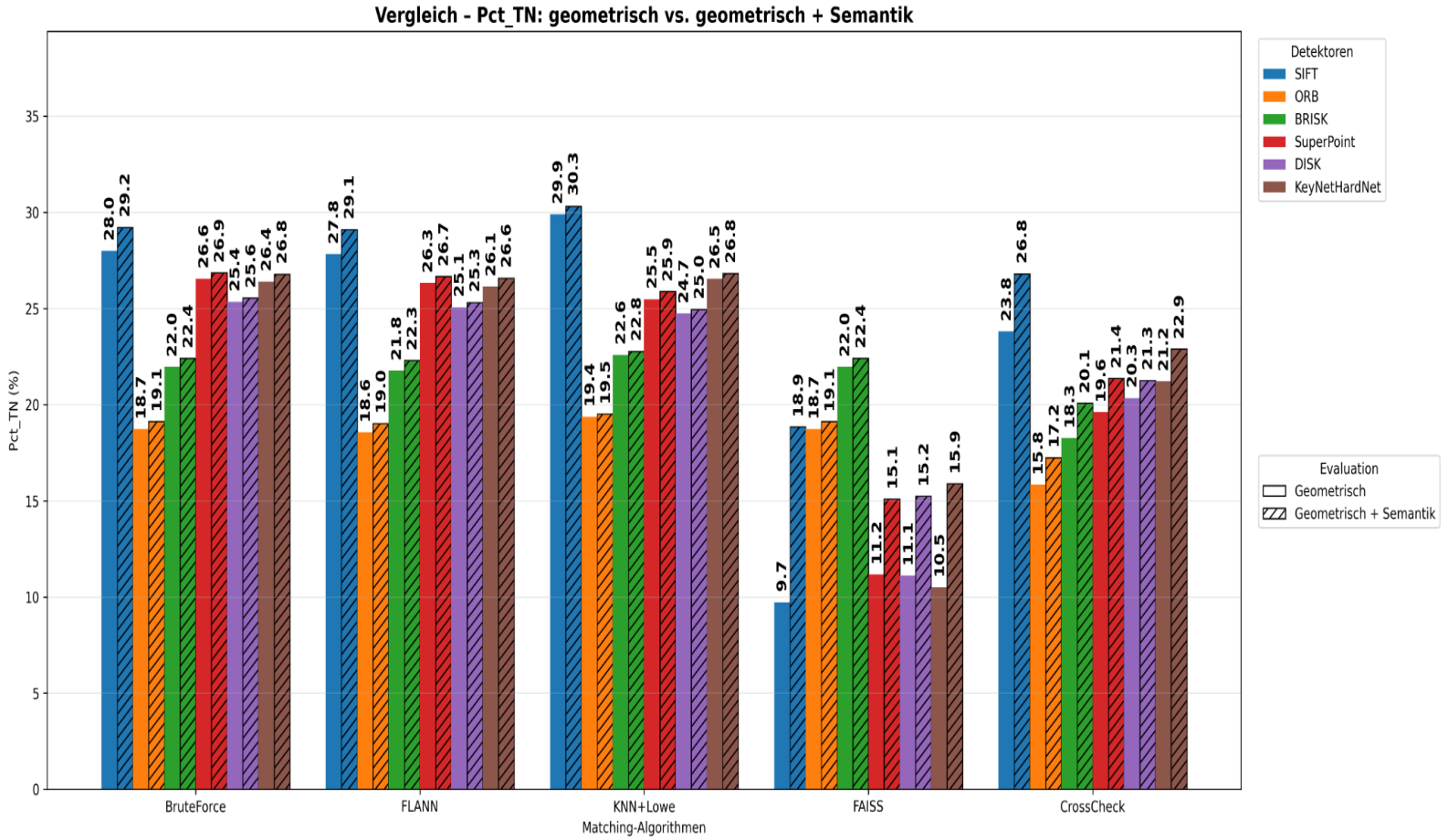
Folge der strengeren Filterung



- False Negatives steigen häufig an
- Einige geometrisch korrekte Matches werden nicht mehr gezählt
- Deshalb sinkt der Recall

Experiment 4: True Negatives

Warum steigt Accuracy nur teilweise?



- True Negatives steigen in einigen Kombinationen
- Nur in 9 Fällen ist dieser Gewinn größer als der TP-Verlust
- Deshalb verbessert sich die Accuracy nur in diesen Fällen

Backup: SIFT (Scale-Invariant Feature Transform)

- **Skalenraum aufbauen**
  - Das Bild wird mehrfach geglättet und in verschiedenen Größen betrachtet
- **Keypoints finden**
  - Punkte liegen häufig an Ecken, Kanten oder markanten Strukturen
  - Dafür wird die Difference-of-Gaussians-Methode verwendet
- **Instabile Punkte entfernen**
- **Orientierung bestimmen**
  - Grundlage sind die lokalen Gradienten um den Keypoint
- **Deskriptor berechnen**
  - Lokale Gradientenrichtungen werden in Histogrammen zusammengefasst
  - Daraus entsteht ein 128-dimensionaler Deskriptor
- **Vorteile**
  - Sehr robust gegenüber Skalierung und Rotation
  - Starker und aussagekräftiger Deskriptor
  - Gute Referenzmethode für Feature Matching
- **Nachteile**
  - Langsamer als binäre Verfahren wie ORB oder BRISK.
  - Höherer Speicherbedarf durch 128-dimensionalen Gleitkomma-Deskriptor.

Backup: ORB (Oriented FAST and Rotated BRIEF)

- **Keypoints finden**
  - FAST vergleicht Helligkeitswerte um einen Kandidatenpunkt
  - Wenn sich ein Punkt stark von seiner Umgebung unterscheidet, wird er als Keypoint erkannt
- **Stärkste Punkte auswählen**
  - Nur die stärksten und stabilsten Punkte werden weiterverwendet
- **Orientierung bestimmen**
  - Dafür wird die Helligkeitsverteilung in der Umgebung genutzt
- **Deskriptor berechnen**
  - ORB nutzt einen BRIEF-ähnlichen binären Deskriptor
  - Dabei werden Pixelpaare in der Umgebung verglichen und entsteht ein Bit: 0 oder 1
- **Vorteile**
  - Sehr schnell
  - Speicherarm
  - Gut für Echtzeitanwendungen
  - In meinen Ergebnissen sehr effizient auf der CPU
- **Nachteile**
  - Weniger robust als SIFT bei starken Perspektiv- oder Beleuchtungsänderungen
  - Binärer Deskriptor ist oft weniger diskriminierend als SIFT

Backup: BRISK (Binary Robust Invariant Scalable Keypoints)

- **Keypoints in mehreren Skalen finden**
  - BRISK betrachtet das Bild in verschiedenen Auflösungen
  - Dadurch können Merkmale erkannt werden, auch wenn Objekte größer oder kleiner erscheinen
- **Orientierung bestimmen**
  - BRISK nutzt ein kreisförmiges Abtastmuster um den Keypoint
  - Weit entfernte Punktpaare helfen, die dominante Richtung zu berechnen
  - Dadurch wird das Verfahren rotationsrobuster
- **Deskriptor berechnen**
  - Für den Deskriptor werden nahe Punktpaare im Kreis verglichen
  - Die Helligkeitswerte dieser Punktpaare werden miteinander verglichen
  - Daraus entsteht ein binärer Deskriptor
- **Vorteile**
  - Schnell und speichereffizient
  - Robuster gegenüber Skalierung als ORB
  - Geeignet für effiziente Anwendungen
- **Nachteile**
  - Meist weniger diskriminierend als SIFT
  - Bei sehr schwierigen Bildveränderungen oft schwächer als lernbasierte Verfahren

Backup: SuperPoint

- **Bild wird durch ein CNN verarbeitet**
- **Keypoint-Karte erzeugen**
  - SuperPoint gibt eine Wahrscheinlichkeitskarte aus
- **Deskriptor-Karte erzeugen**
  - Für jeden Keypoint kann daraus ein lokaler Deskriptor entnommen werden
- **Gemeinsames Lernen**
  - Detektion und Beschreibung sind im selben Modell verbunden
- **Training**
  - Zuerst Training auf synthetischen Formen
  - Danach Homographic Adaptation
  - Dadurch lernt das Netzwerk, gleiche Punkte auch bei Perspektivänderungen wiederzufinden
- **Vorteile**
  - Oft robuster als klassische Verfahren
  - Detektor und Deskriptor sind gut aufeinander abgestimmt
  - Profitiert stark von GPU-Beschleunigung
- **Nachteile**
  - Höherer Rechenaufwand als ORB oder BRISK
  - Auf CPU deutlich langsamer

Backup: DISK (Deep Image Similarity Kernel)

- **Bild wird durch ein CNN verarbeitet**
- **Keypoints werden probabilistisch ausgewählt**
  - Die Auswahl der Keypoints ist diskret
- **Optimierung auf Matching-Qualität**
  - DISK lernt nicht nur auffällige Punkte
  - Es lernt Punkte, die zu korrekten Korrespondenzen führen
- **Deskriptoren berechnen**
  - Diese Deskriptoren sollen echte Korrespondenzen gut unterscheidbar machen
- **Vorteile**
  - Sehr starke Matching-Ergebnisse
  - In meinen Ergebnissen oft hohe Precision, Recall, F1 und Accuracy
  - Direkt auf gute Korrespondenzen optimiert
- **Nachteile**
  - Höherer Rechenaufwand
  - GPU ist für effiziente Ausführung wichtig
  - Auf CPU langsamer als klassische Verfahren

Backup: KeyNetHardNet

- **Key.Net findet Keypoints**
  - Key.Net sucht stabile Bildpunkte
  - Es kombiniert klassische Bildfilter mit gelernten CNN-Filtern
  - Dadurch werden bekannte Strukturen wie Ecken und Kanten mit lernbasierten Merkmalen verbunden
- **Stabile Punkte auswählen**
- **HardNet beschreibt die Keypoints**
  - Für jeden Keypoint wird die Umgebung durch HardNet beschrieben
  - HardNet erzeugt einen lernbasierten Deskriptor
- **Training mit schwierigen Negativbeispielen**
  - HardNet lernt, echte Matches nah zusammenzubringen
  - Falsche, aber ähnliche Matches sollen weit auseinanderliegen
- **Vorteile**
  - Robuste Keypoints
  - Starker lernbasierter Deskriptor
  - Gute Qualität bei schwierigen Bildveränderungen
- **Nachteile**
  - Hoher Rechenaufwand
  - Auf CPU deutlich langsamer
  - In meinen Ergebnissen war KeyNetHardNet bei der Extraktionszeit besonders langsam

Backup: BruteForce Matcher

- **Deskriptoren vergleichen**
  - Jeder Deskriptor aus dem ersten Bild wird mit allen Deskriptoren aus dem zweiten Bild verglichen.
- **Distanz berechnen**
  - Für jeden Vergleich wird eine Distanz berechnet
  - Bei SIFT oder CNN-Deskriptoren meistens L2-Distanz
  - Bei ORB oder BRISK meistens Hamming-Distanz
- **Besten Kandidaten auswählen**
  - Der Deskriptor mit der kleinsten Distanz wird als bester Match gewählt
  - Kleine Distanz → Die beiden Deskriptoren sind ähnlich
- **Matching-Ergebnis**
  - Das Ergebnis ist eine Liste von Punktpaaren zwischen zwei Bildern
- **Vorteile**
  - Einfach zu verstehen und umzusetzen
  - Liefert exakte nächste Nachbarn
  - Keine Approximation
- **Nachteile**
  - Langsam bei vielen Keypoints
  - Rechenaufwand steigt stark mit der Anzahl der Deskriptoren

Backup: KNN + Lowe Ratio Test

- **Zwei nächste Nachbarn suchen**
  - Für jeden Deskriptor aus Bild 1 werden die zwei ähnlichsten Deskriptoren aus Bild 2 gesucht
  - Es gibt also einen besten und einen zweitbesten Kandidaten
- **Distanzen vergleichen**
  - Danach wird geprüft, wie viel besser der beste Match im Vergleich zum zweitbesten ist
  - Wenn beide Distanzen sehr ähnlich sind, ist der Match unsicher
- **Lowe Ratio Test**
  - Ein Match wird nur akzeptiert, wenn der beste Kandidat deutlich besser ist als der zweitbeste
  - Dadurch werden mehrdeutige Matches entfernt
- **Matching-Ergebnis**
  - Es bleiben nur Matches übrig, die relativ eindeutig sind
  - Besonders bei wiederholenden Strukturen ist das wichtig
- **Vorteile**
  - Filtert viele unsichere und mehrdeutige Matches
  - Oft bessere Precision als einfaches BruteForce
  - Besonders bekannt durch SIFT
- **Nachteile**
  - Es können auch korrekte Matches entfernt werden
  - Dadurch kann der Recall sinken
  - Etwas mehr Aufwand als einfaches Nearest-Neighbor-Matching

Backup: FLANN

- **Indexstruktur aufbauen**
  - FLANN vergleicht nicht einfach jeden Deskriptor mit allen anderen
  - Stattdessen wird zuerst eine Suchstruktur aufgebaut
  - Zum Beispiel KD-Trees oder andere Indexmethoden
- **Schnelle Suche**
  - Bei einer Anfrage durchsucht FLANN nur relevante Teile des Indexes
  - Dadurch ist die Suche schneller als vollständiges BruteForce-Matching
- **Approximation**
  - FLANN findet nicht immer garantiert den exakt besten Nachbarn
  - Es findet aber sehr schnell einen sehr ähnlichen Kandidaten
  - Es ist also eine approximative Suche
- **Matching-Ergebnis**
  - Die gefundenen Nachbarn werden als Matches verwendet
- **Vorteile**
  - Sehr schnell
  - Gut geeignet für große Deskriptormengen
  - In meinen Ergebnissen oft der schnellste Matcher
- **Nachteile**
  - Nicht immer exakt
  - Kann bei manchen Deskriptoren schlechtere Qualität liefern

Backup: FAISS

- **Deskriptoren als Vektoren speichern**
  - Alle Deskriptoren werden als Vektoren betrachtet
  - Ziel ist, für jeden Query-Vektor den ähnlichsten Vektor zu finden
- **Index erstellen**
  - FAISS baut eine Suchstruktur für diese Vektoren auf
  - Es gibt einfache exakte Indizes und schnelle approximative Indizes
- **Nächste Nachbarn suchen**
  - Für jeden Deskriptor aus Bild 1 sucht FAISS den nächsten Nachbarn in Bild 2
  - Die Suche kann sehr effizient durchgeführt werden
- **Matching-Ergebnis**
  - Die ähnlichsten Vektoren werden als Matches ausgegeben
- **Vorteile**
  - Sehr effizient bei großen Vektormengen
  - Gut für CNN-Deskriptoren
  - Kann hohe Recall-Werte erreichen
- **Nachteile**
  - Je nach Index nicht immer exakt
  - Einrichtung und Parameter können komplexer sein
  - Für kleine Datenmengen nicht immer notwendig

Backup: CrossCheck Matcher

- **1. Matching von Bild 1 zu Bild 2**
  - Zuerst wird für einen Deskriptor aus Bild 1 der beste Deskriptor in Bild 2 gesucht.
- **2. Matching von Bild 2 zu Bild 1**
  - Danach wird in die Gegenrichtung geprüft.
  - Der Deskriptor aus Bild 2 muss wieder denselben Deskriptor aus Bild 1 als besten Partner finden.
- **3. Gegenseitige Bestätigung**
  - Ein Match wird nur akzeptiert, wenn beide Richtungen übereinstimmen.
- **4. Unsichere Matches entfernen**
  - Einseitige oder instabile Matches werden verworfen.
  - Dadurch wird die Matchliste strenger und oft sauberer.
- **Vorteile**
  - Reduziert viele falsche Matches.
  - Liefert konsistentere Zuordnungen.
  - Einfaches Prinzip.
- **Nachteile**
  - Entfernt auch einige korrekte Matches.
  - Dadurch kann der Recall sinken.
  - Oft langsamer, weil zwei Richtungen geprüft werden.
  - In meinen Ergebnissen war CrossCheck häufig laufzeitintensiv.